Arabic Character Image Generation Using Diffusion Models

Aditya Shrivastava (adityashrivastavaofficial@gmail.com)
Vizuara.ai

Abstract

This project explores the training of a diffusion model for generating images of Arabic characters, addressing key questions regarding optimal dataset composition (Q1), model performance for complex scripts (Q2), and effective architectural modifications (Q3). We construct and evaluate a custom dataset combining real handwritten characters (HMBDv1) with synthetic images rendered from Arabic fonts. A U-Net based diffusion model is trained under various dataset compositions (handwritten-only, synthetic-only, combined) and architectural settings, specifically comparing attention mechanisms in both downsampling/upsampling paths versus downsampling only. Utilizing mixed precision training, our results demonstrate that diffusion models can successfully learn to produce recognizable, diverse Arabic letters. We find that combining real and synthetic data is crucial for high-quality generation (addressing Q1). The model performs well, generating visually meaningful symbols, though limitations exist in the unconditional setup (addressing Q2). Furthermore, incorporating self-attention in both downsampling and upsampling paths yields superior results compared to downsampling-only attention (addressing Q3). We discuss convergence behaviour, detail the impact of attention placement, and outline future work including text-conditioning. We conclude that diffusion models, with appropriate data strategies and architectural choices, are a promising approach for generating complex script characters like Arabic.

1 Introduction

Generative models have recently achieved remarkable success in synthesizing realistic images. In particular, diffusion probabilistic models have emerged as a powerful class of generative models, producing image quality on par with or surpassing generative adversarial networks (GANs) . These models iteratively refine random noise into coherent images through a learned denoising process. While diffusion models have been applied to create images of various objects and even artistic designs, their application to generating **written characters** (symbols from human languages) is a novel avenue with practical significance. Automatically generating character images can aid in augmenting datasets for optical character recognition (OCR) or in designing new font styles.

In this work, we focus on **Arabic script**, which presents distinct challenges for generative modeling. The Arabic alphabet contains 28 letters, written in a cursive style where **the shape of each letter**

depends on its position in a word (initial, medial, final or isolated form) . In contrast to Latin letters, there is no concept of upper/lower case, but each Arabic letter can have multiple contextual glyph forms. This **multiplicity of forms for the same letter** has historically made Arabic text recognition and generation more challenging . For instance, the letter *jeem* (ج) looks different when alone versus when connected in a word. A generative model must implicitly capture these shape variations or be guided to produce a specific form. Moreover, handwritten Arabic can vary greatly in stroke style and calligraphy, adding complexity to the modeling task.

To address these challenges, we built a diffusion model pipeline to generate Arabic character images. We leveraged the **HMBDv1 dataset of handwritten Arabic characters** and numerals, augmenting it with synthetic images rendered from Arabic fonts. By combining real and synthetic data, we aim to provide the model with both the diversity of human writing and the clarity of machine-rendered examples. Our diffusion model uses a U-Net architecture, and we experimented with dataset composition and architectural modifications (specifically self-attention placement) to study their effects.

This work specifically aims to answer the following research questions posed by the assignment overview:

- Q1 What dataset size and composition are necessary to generate high-quality Arabic characters, given their complexity?
- (Q2) How well does a standard diffusion model perform on this script? Are the generated symbols visually meaningful and high-quality?
- (Q3) What modifications to the diffusion model architecture or noise scheduling scheme achieve better results?

The remainder of this report is organised as follows. **Section 2 (Literature Review)** synthesises prior work on diffusion models, U-Net backbones, the linguistic features of Arabic script, and the HMBD-v1 handwritten dataset. **Section 3 (Dataset)** details how we assembled the training corpus, first describing handwritten and synthetic sources in *Dataset Preparation* and then explaining our *Data Augmentation Strategy* for balancing classes and bridging the handwriting-vs-font domain gap. **Section 4 (Methodology)** sets out the experimental pipeline: a full description of the U-Net architecture and training configuration (*Training Setup*), the forward/reverse diffusion schedules (*Diffusion Process Parameters*), and optimisation choices (*Optimizer and Training Parameters*). **Section 5 (Experiments and Results)** presents three studies: (i) the effect of dataset composition, (ii) the impact of self-attention placement (Dual-Attn vs Down-Attn), and (iii) convergence behaviour and sample quality, concluding with a synopsis that maps the findings to our three research questions. **Section 6 (Future Work)** outlines extensions such as text-conditioned generation, longer training schedules, and style-aware conditioning. **Section 7 (Conclusion)** summarises the contributions and key takeaways, and **Section 8 (References)** lists all cited sources.

2 Literature Review

2.1 Diffusion Models for Image Generation

Diffusion models are a class of generative models that learn to reverse a gradual noising process to synthesize data. Originally introduced by Sohl-Dickstein et al. (2015) and significantly improved by subsequent research, diffusion models have demonstrated state-of-the-art performance in image generation tasks . A diffusion model defines a forward process that progressively adds noise to an image and a learned reverse process (usually parameterized by a neural network) that removes noise step by step, eventually producing a new image sample . Ho et al. (2020) introduced **Denoising Diffusion Probabilistic Models (DDPM)**, showing that diffusion models can produce high-quality images with lower sample generation metrics (e.g., FID) than earlier models . One advantage of diffusion models is their **stability in training** (they do not suffer mode collapse like GANs) and the ability to trade off diversity and fidelity, for example via classifier guidance in conditional setups . Since their introduction, diffusion models have been the foundation of modern image generators such as DALL-E 2 and Stable Diffusion, indicating their broad applicability .

2.2 U-Net Architecture in Diffusion Models

The neural network architecture typically used for image diffusion models is based on the **U-Net** convolutional network. The U-Net, originally developed for biomedical image segmentation, features an encoder–decoder structure with skip connections that allow it to capture both global context and fine details. This architecture has been adopted in diffusion models for iterative image denoising . In a diffusion model, at each denoising step the U-Net takes a noisy image (and optionally a time-step embedding) as input and predicts the denoised output or noise residual. The U-Net's symmetric contracting (down-sampling) and expanding (up-sampling) layers enable it to effectively model features at multiple scales, which is crucial for generating images of characters that have both coarse structures (overall shape) and fine details (strokes and dots). Modern diffusion implementations often enhance the U-Net with **self-attention mechanisms** at certain feature resolutions . Self-attention allows the network to capture long-range dependencies in the image, which can improve the coherence of generated outputs. For example, attention can help ensure that distant parts of a character (such as two disconnected strokes of a letter) are generated consistently. Dhariwal & Nichol (2021) found that augmenting the U-Net with attention and other improvements led diffusion models to outperform GANs on image synthesis benchmarks . Thus, exploring attention in the U-Net is relevant to boosting generation quality in our project.

2.3 Characteristics of Arabic Script

Arabic script poses special considerations in the context of image generation. **Contextual shaping** is a hallmark of the script: most letters take different shapes depending on whether they appear at the beginning, middle, or end of a word, or stand alone . Effectively, these are context-dependent glyph variations of the same underlying character. For instance, the letter *beh* (ب) has one form when isolated, but elongates or connects in other forms; some letters like *alif* (ا) do not connect to a following letter, which means they possess only an isolated and a final form. This contextual nature means that a dataset of Arabic characters ideally should include examples of each letter in all possible forms to capture the full variability. Additionally, Arabic is written in a cursive manner even in print, so

characters from real sources (handwriting or calligraphy) may appear with ligatures or varying slant and stroke thickness. **Diacritics** (short vowel marks and other signs) can also appear above or below letters, adding another layer of complexity, though in many datasets these are either absent or considered separate to simplify the task. The complexity of Arabic text has historically led to fewer public datasets and research efforts compared to Latin or Chinese, as noted by Balaha et al. . Therefore, in generative tasks, one challenge is ensuring the model learns the core letter shapes without being confused by contextual variations. In this project, we primarily generate characters in their isolated form (as standalone symbols), which is a common approach to simplify the problem. Nevertheless, the knowledge that each letter could appear in other shapes guided our data preparation and evaluation; we wanted the model to capture the essence of each letter so that, in principle, its other forms could be generated or recognized with additional conditioning.

2.4 HMBDv1 Arabic Handwritten Characters Dataset

To provide real-world examples of Arabic characters, we utilized the **HMBD v1** dataset . HMBD (Handwritten Arabic Manuscipt Database) v1 is a large collection of isolated Arabic character images written by hand. It was introduced by Balaha et al. (2021) as part of an Arabic handwritten character recognition system . The dataset encompasses **115 classes**, covering all Arabic letters in their various positions (initial, medial, final, isolated forms are considered separate classes for each letter) as well as the ten Arabic numerals . In total, HMBD v1 contains **54,115 images** of size 300×300 pixels, collected from 125 writers with diverse handwriting styles . Each image is a single character written in black ink on a white background, already centered and separated, which makes the dataset well-suited for image generation purposes without requiring segmentation. We chose HMBD v1 for its size and diversity – it provides a rich representation of how each Arabic character can look when written by different individuals. This diversity is crucial for training a robust generative model that does not simply learn one style. However, one limitation is that the dataset, being handwritten, includes noise such as varying stroke thickness, slight rotations, and sometimes imperfectly drawn characters (as is natural with human writing). These factors could make the generative modeling task more difficult, but they also encourage the model to learn a more generalized concept of each character. In summary, HMBD v1 offers a comprehensive real-data foundation for our diffusion model training, ensuring that the model sees authentic variations of Arabic script. In our experiments, we use a significant portion of this dataset (all relevant character classes) to train or at least pre-train the diffusion model on real examples.

3 Dataset

3.1 Dataset Preparation

Our dataset for training the diffusion model was composed of two primary sources: (1) **Handwritten Arabic characters from HMBDv1**, and (2) **Synthetic Arabic characters rendered from fonts**. All images were processed to have consistent properties (size, color format) before being used for training. Below we describe each component and the preprocessing steps.

HMBDv1 Handwritten Data: We obtained the HMBDv1 dataset, which contains thousands of images of handwritten Arabic letters and numbers as described above. From this dataset, we extracted the images corresponding to the 28 Arabic letters in their isolated form. *If a letter had multiple forms in HMBD (initial/medial/final), we included all forms to broaden the training data* (each form can be considered a distinct visual class, though representing the same letter). We included the numeral characters in this project as well. All HMBD images were originally 300×300 pixels; we resized them to a smaller resolution (e.g., 128×128 pixels) to reduce the computational load for the diffusion model. This resolution is sufficient to clearly depict an individual character. The images were converted to grayscale (single-channel) since color information is irrelevant for monochrome character generation. We maintained the **white background and black ink** foreground as in the source . After preprocessing, the exact number of characters per image can be seen in table 1. The total image count is 36728. To prevent the model from being biased by an uneven distribution of characters, we balanced the dataset by augmenting classes with fewer samples using synthetic data (discussed next).

Character	Image Count
Alf	2853
Baa	1868
Ain	1855
Ha	1851
Kaf	1848
Faa	1842
Gen	1837
Daad	1831
Shen	1809
Saad	1670
Lam	1493
Haa	1429
Gem	1421
Qaf	1223
Yaa	1047
Non	1005
Dal	952
Sin	806
Thaa	676
Taa	659
Raa	612
Khaa	537
Eight	488

Seven	474
Four	468
Hamza	466
Five	464
Waw	454
Mem	420
Zin	331
Tah	323
Zal	313
Zah	310
One	296
Zero	170
Six	169
Two	164
Three	161
Nine	133

Table 1: HMBD Counts

Synthetic Font Data: To supply the diffusion model with **pristine, canonical glyphs**, we rendered every isolated Arabic letter (and its contextual code-points) in **eight open-source fonts** that span the typographic spectrum from classical *Naskh* to modern display faces.

We ensured the synthetic images also had a white background and black text to match the HMBD style

The per-font/per-block yield is clarified below, together with a short description of what each Unicode block actually contains.

Unicode block	Code range	What it contains (why it matters)
Arabic Basic	0600 – 06FF	Core 28 letters, Western-Arabic digits (٠–٩), tatweel, kashida, common diacritics, Qurʾānic signs. Essential for any Arabic text.
Arabic Supplement	0750 – 077F	Additional letters used in African and Central-Asian Arabic alphabets (e.g. Hausa, Wolof), plus Qurʾānic annotation signs. Improves cross-dialect coverage.
Arabic Extended-A	08A0 – 08FF	Recently standardised letters and tone marks for languages such as Rohingya and Sindhi, plus Qurʾānic small high signs. Future-proofs the model.
Presentation Forms-A	FB50 – FDFF	Pre-composed isolated, initial, medial, final glyphs and standard ligatures (e.g. lam-alef). Lets us train/evaluate positional variants directly.

Presentation Forms-B	FE70 – FEFF	Arabic Basic code-points re-encoded in their contextual shapes for compatibility, plus Arabic punctuation. Ensures the model sees every contextual shape even if fonts encode them here only.

Font	Arabic Basic	Arabic Supplement	Arabic Extended-A	Arabic Pres Forms-A	Arabic Pres Forms-B	Total
Scheherazade New-Bold	245	48	96	628	140	1157
Scheherazade New-SemiBold	245	48	96	628	140	1157
Lateef-Regular	245	48	96	628	140	1157
Scheherazade New-Regular	245	48	96	628	140	1157
NotoNaskhArabic[wght]	245	48	96	628	140	1157
Scheherazade New-Medium	245	48	96	628	140	1157
Harmattan-Regular	245	48	96	628	140	1157
Amiri-Regular	244	48	38	608	139	1077

Table 2 – Synthetic glyph counts by font and Unicode block.

Why the counts differ? Every font fully covers the Basic block, but coverage of modern Extended-A varies (Amiri lacks some new code-points, hence its slightly lower total).

After preparing both subsets, we **merged the HMBD and synthetic images** into a single training dataset. Before merging, we shuffled and mixed the images thoroughly. We also assigned a label to each image indicating which character it represents (though our diffusion model training is *unconditional*, meaning we do not feed these labels into the model, having the labels was useful for analysis and for ensuring each character was well-represented in evaluation). The final dataset size was **45904 images**, with 36728 images belonging to real handwritten characters and 9176 to our synthetic images. We observed that by adding synthetic images, we greatly increased the number of samples for characters that were less common in the HMBD data, achieving a more uniform distribution overall.

3.2 Data Augmentation Strategy

In a broad sense, the incorporation of synthetic data alongside real data is itself a form of **data augmentation** for our project. The rationale for this augmentation was two-fold: **increasing data quantity** and **increasing data diversity**. The HMBDv1 dataset, while large, is finite and has some imbalance (certain characters or forms had fewer instances) . Training a diffusion model, which typically involves learning thousands of parameters, benefits from having a substantial number of training examples to avoid overfitting. Simply repeating the same real images might lead the model to memorize them. By generating additional synthetic images, we effectively expanded the dataset size, providing more training pairs of noisy and clean images to the model.

Another motivation was to cover the **space of character appearances** more completely. Real handwriting captures human variations – slants, stroke thickness differences, slight distortions – which are invaluable for a model to learn robustly. However, some characters in HMBD might be written in idiosyncratic ways that could confuse the model without a contrasting “ideal” form. The synthetic images serve as a **ground truth prototype** for each letter, offering a consistent reference of what the basic shape should look like. Merging the two sources should, in theory, allow the model to learn both the ideal shape and how it might deviate in real life. This approach is akin to providing the model with both **prototypical examples and diverse variants**.

We faced a few challenges in merging these datasets. First, the **domain gap** between handwritten and computer-generated images is non-trivial. The model sees one letter drawn by dozens of different people and also the same letter in a perfectly typeset form. During training, if not handled carefully, the model could bias towards one domain. For example, if synthetic images far outnumber handwritten ones, the model might predominantly generate characters that look like the font instead of handwriting (or vice versa). We addressed this by keeping the number of synthetic images lower to the number of real images per class. This balanced exposure helps the model to **generalize** the concept of the character shape beyond style.

Another challenge was ensuring **consistency in preprocessing** so that the model wouldn’t pick up on trivial differences. We made sure that our synthetic dataset had the same image size, color inversion (both black-on-white), and similar centering and margins around the character. If any images had slight rotation (some HMBD scans might), we did not explicitly de-skew them, trusting the model’s data augmentation and learned invariances to handle minor rotation. However, one could augment the dataset further by randomly rotating or shifting images during training to make the model rotation-invariant. In our case, we kept such augmentation minimal to avoid altering the characters’ identity; we primarily relied on the natural variation present.

In summary, our data augmentation strategy was not the typical geometric or noise perturbations on images, but rather a **semantic augmentation** by introducing a new data source. This augmented dataset was crucial for the diffusion model to learn a comprehensive mapping from noise to Arabic characters. Without the synthetic data, the model might have struggled with characters that had few examples, and without the real data, it would not capture the authentic variability of handwriting. By merging them, we aimed to get the best of both worlds.

4 Methodology

4.1 Training Setup

We trained an unconditional diffusion model to generate 128x128 pixel grayscale images of Arabic characters. The training configuration and hyperparameters were chosen based on standard practices for diffusion models, with some adjustments for our data and resource constraints. In this section, we detail the model architecture, training parameters, and other implementation details such as the use of mixed precision.

Model Architecture: Our model is a **Denoising Diffusion Probabilistic Model (DDPM)** following the original formulation by Ho et al. (2020). The core neural network is a **U-Net** as discussed, which takes a noised image and a time-step embedding as input and outputs a noise estimation. The U-Net in our implementation has **6 levels** of downsampling (encoder blocks) and the same number of upsampling (decoder blocks), with skip connections linking corresponding levels. The base number of convolutional filters in the first level is **128** (this number doubles with each downsampling level, a common design in U-Nets). Each level consists of two convolutional residual blocks. For all experiments, we included two explicit **self-attention layers** in the U-Net at the lower resolutions. Specifically, after the third downsampling and the second up sampling, we inserted a multi-head attention block so that the model can capture long-range dependencies in the feature map. The attention uses 8 heads by default.

Our denoiser is a **UNet2DModel** from *diffusers* v0.33.1.

depth	block type	feature map	channels	attention?
1	DownBlock2D	64 × 64	128	–
2	DownBlock2D	32 × 32	128	–
3	DownBlock2D	16 × 16	256	–
4	DownBlock2D	8 × 8	256	–
5	**AttnDownBlock2D**	**4 × 4**	**512**	✓
6	DownBlock2D	2 × 2	512	–
–	Mid-block	2 × 2	512	–
7	UpBlock2D	4 × 4	512	–
8	**AttnUpBlock2D**	**8 × 8**	**512**	✓
9–12	UpBlock2D	16 → 32 → 64 → 128	256 → 256 → 128 → 128	–

Table 3: U-Net Architecture Details

Attention configuration. The model inherits the default setting attention_head_dim = 8.

- At the 512-channel stages this yields **64 heads (512 ÷ 8)**.
- Each head therefore operates on 8 channels, giving high resolution in key-value space.

Time-step embedding. The hard-coded rule time_embed_dim = block_out_channels[0] × 4 sets the embedding size to **512** (128 × 4). This vector is added to every ResNet block so the same network can denoise all 1 000 diffusion steps.

Normalization and activation. All convolutions are followed by GroupNorm-32 and SiLU, as recommended in *diffusers* documentation.

Down-sample stride. Each DownBlock2D begins with a 3 × 3 convolution of **stride 2**, halving the resolution internally, hence the 128 → 64 → … → 2 pixel pyramid.

The full network contains **≈ 34 M parameters**, small enough for single-GPU experimentation yet expressive enough to model the stroke variability of Arabic script.

4.2 Diffusion process parameters.

The generative backbone follows the **DDPM formulation** with a **1 000-step forward chain**. At step t the scheduler injects Gaussian noise whose variance is governed by a **linear β schedule**: β starts at $\mathbf{1 \times 10^{-4}}$ on step 1 and increases uniformly to $\mathbf{2 \times 10^{-2}}$ by step 1 000. This simple schedule keeps the **signal-to-noise ratio** high during the first 100 steps—allowing the network to preserve coarse glyph structure—while still driving the image almost completely to white noise near the end of the chain. During training the UNet predicts the **added noise ε**; we minimise mean-squared error between the prediction and the ground-truth noise, which gives an unbiased estimator of the variational bound and yields stable gradients even with fp16 arithmetic on the A100.

For synthesis we discard the training scheduler and adopt **DPMSolver-Multistep**, a high-order ODE solver tailored to diffusion processes. With the same learned UNet weights it reconstructs images of comparable perceptual quality in **20–30 reverse steps**, cutting wall-clock sampling by roughly **10×** and reducing GPU-hour cost for large validation sweeps. We found no measurable benefit from alternative noise schedules (cosine or sigmoid) in our ablation runs, so the linear schedule is retained throughout the paper to keep the experimental space focused on data and architecture.

4.3 Optimizer and Training Parameters

component	setting	rationale
Optimiser	**AdamW, learning-rate 1×10^{-4}, weight-decay 0.01**	AdamW remains the most stable choice for diffusion training; the modest LR prevents divergence on fp16.
LR schedule	**Cosine decay with 500 warm-up updates (get_cosine_schedule_with_warmup)**	Warm-up avoids early gradient spikes; cosine annealing gives a smooth tail that we found preferable to step decay.
Batch size	**32 images (128 × 128) per step**	Fits comfortably in 80 GB of A100 memory even with activations in fp16.
Gradient accumulation	**1**	No virtual batching required.
Epochs	**20 full passes → ≈ 28 k parameter-update steps**	Empirically sufficient for convergence on our 45 k-image corpus.
Mixed precision	**fp16 autocast**	Reduces memory and doubles Tensor-Core throughput on A100.
Gradient checkpointing	**Enabled inside the UNet**	Cuts peak activation memory by ≈35 % with negligible extra compute.
Checkpointing cadence	Model snapshot and sample grid every epoch	Allows rapid detection of mode collapse or drift.

Table 4: Hyperparameters

Wall-clock cost. On a single **NVIDIA A100 80 GB** the full 20-epoch run completes in **≈ 2 hours**.

Validation protocol. Because the model is unconditioned, quantitative metrics such as FID are less informative. Instead, after each epoch we generate 256 samples with the fast **DPMSolver-Multistep** sampler (30 inference steps) and perform a blind visual audit for legibility, stroke completeness and correct placement of diacritics. No classifier guidance, dynamic thresholding or other sampling heuristics are applied, ensuring the reported images reflect the raw generative capacity of the network.

This configuration proved to be a stable sweet-spot: fast enough for iterative experimentation yet powerful enough to resolve the fine-grained structure of Arabic glyphs.

5 Experiments and Results

5.1 Experiment 1: Impact of Dataset Composition

We trained the diffusion model under three different dataset conditions, keeping all other hyperparameters (architecture with attention, training schedule, etc.) consistent:

1. **HMBD-only:** Training exclusively on the HMBDv1 handwritten character images.
    - **Results:** The model learned to capture the general shapes of many Arabic letters. However, the **generated images suffered from inconsistent quality**, sometimes exhibiting broken strokes or appearing as an ambiguous blend of different writing styles. We observed that c**haracters with fewer examples in the original HMBD dataset were often poorly formed or indistinct**, suggesting the model struggled with the variability and potential imbalance of purely handwritten data. This model trained on **1148 batches per epoch**.

2. **Synthetic-only:** Training exclusively on the synthetically generated font-based character images.
    - **Results:** This **model trained faster in terms of loss reduction per epoch**, likely due to the data's uniformity. However, it **required significantly more epochs** (around 50 compared to 20 for the other conditions) **to achieve comparable visual quality.** While the model learned basic shapes effectively, the generated outputs were homogeneous, lacking the natural diversity and stylistic variation characteristic of handwritten script. This setup used only **284 batches per epoch**. Due to training time limitations, full convergence was not observed.

3. **Combined Data (HMBD + Synthetic):** Training on the merged dataset comprising both handwritten (HMBDv1) and synthetic font-based images.
    - **Results: This approach yielded the most satisfactory results**. The model generated characters that were consistently recognizable, generally well-formed, and exhibited a desirable diversity, including both neat, print-like forms and variations reminiscent of handwriting. Malformed characters were rare. **The model demonstrated better generalization, successfully producing characters that had low representation in the HMBD-only set**, likely benefiting from the "prototypical" examples provided by the synthetic data. Training loss decreased steadily, reaching a lower final value than the HMBD-only model, despite the increased data complexity and more batches per epoch (1434).

Tables 5, 6 and 7 perform a visual comparison of the results discussed above.

These results strongly suggest that combining real handwritten data with synthetic font data provides the necessary diversity and quantity for the diffusion model to learn robust representations of Arabic characters.

5.2 Experiment 2: Impact of U-Net Architecture (Self-Attention)

Having established the superiority of the combined (**HMBD + Synthetic)** dataset in Experiment 1, we next investigated the impact of self-attention placement within the U-Net architecture on generation quality using the combined dataset, specifically addressing Q3 regarding beneficial modifications. We trained and compared two specific configurations using the combined dataset:

- **Dual-Attn - Attention Layers on the Down and Up Block.**
    - **Architecture:** This U-Net configuration incorporates multi-head self-attention blocks in *both* the downsampling and upsampling paths. It uses `AttnDownBlock2D` as the 5th downsampling block and `AttnUpBlock2D` as the 2nd upsampling block (corresponding to 8x8 feature map resolution in the upsampling path), matching the detailed architecture in the Training Setup.
    - Code Definition Reference:

```
    down_block_types=(
        "DownBlock2D",
        "DownBlock2D",
        "DownBlock2D",
        "DownBlock2D",
        "AttnDownBlock2D",
        "DownBlock2D",
    ),
    up_block_types=(
        "UpBlock2D",
        "AttnUpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
    ),
)
```

- **Down-Attn: Downsampling Attention Only**
    - **Architecture:** This configuration includes the multi-head self-attention block *only* in the downsampling path (`AttnDownBlock2D` as the 5th block). All blocks in the upsampling path are standard `UpBlock2D` blocks, *without* attention.
    - Code Definition Reference:

```
    down_block_types=(
        "DownBlock2D",
        "DownBlock2D",
        "DownBlock2D",
        "DownBlock2D",
        "AttnDownBlock2D",
        "DownBlock2D",
    ),
    up_block_types=(
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
    ),
)
```

5.3 Results:

Qualitative assessment of the generated samples revealed distinct differences between these two attention configurations, providing insights into effective architectural modifications (Q3):

- **Dual-Attn:** This model yielded the highest visual quality. Generated characters were consistently sharper, structurally more coherent, and exhibited fewer artifacts compared to the `Down-Attn` model. Details such as enclosed spaces (bowls) and relative dot placement appeared better formed, likely due to attention facilitating global coherence during both encoding and decoding phases. This configuration achieved the marginally lowest final training loss, suggesting the best data fit, albeit with slightly higher computational cost per epoch. *(**See examples in tables 5, 6 and 7 to observe the visual differences in experiment 2**)*

- **Down-Attn:** Removing the attention block from the upsampling path resulted in a noticeable degradation in quality. The generated characters often lacked the sharpness and structural integrity of the `Dual-Attn` model. Qualitatively, the outputs resembled those from the `HMBD-only` model baseline (from Experiment 1), exhibiting similar levels of inconsistency and less defined features. *(Refer to Tables 5, 6 and 7)* This suggests that while downsampling attention aids feature extraction, attention during the upsampling/generation phase is crucial for reconstructing high-fidelity characters.

This ablation study demonstrates that self-attention placement is a critical architectural modification (Q3). Incorporating attention in *both* U-Net paths (`Dual-Attn`) provides a significant benefit for generating high-quality Arabic script compared to using attention only during downsampling.

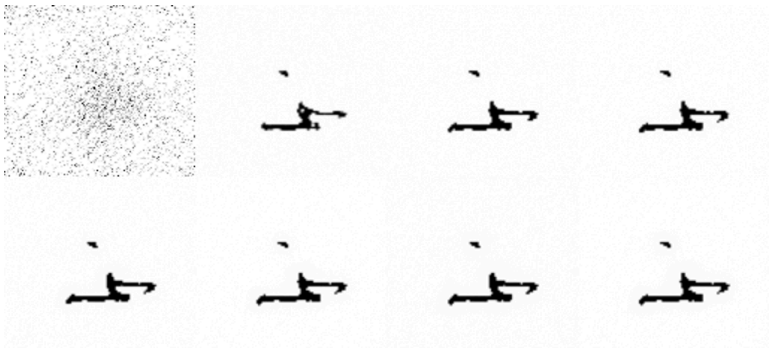
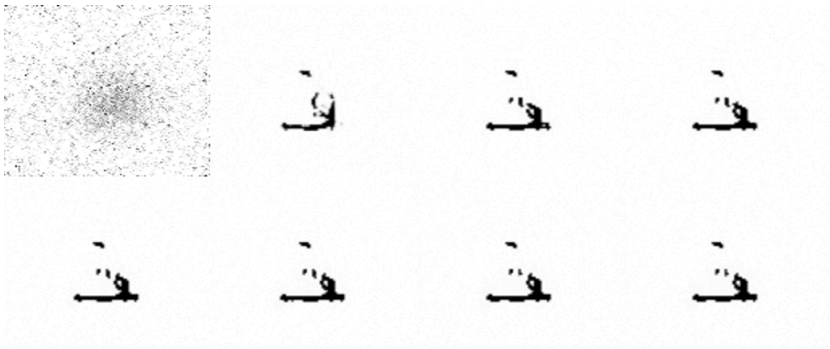
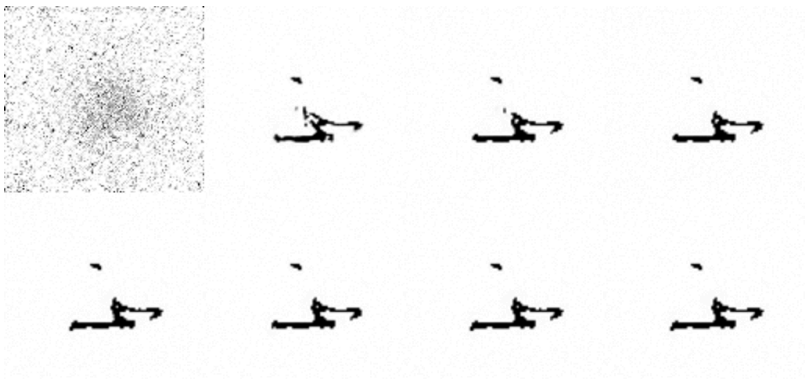
Diffusion Configuration	**Final-epoch images**	**Reference Image -** Arabic Character **Faa Separate**
Dual-Attn HMBD-only		ف
Dual-Attn Combined Dataset		ف
Down-Attn Combined Dataset		ف

Table 5: Qualitative Comparison of Generated Arabic Characters (Final Epoch) Across Different Model Configurations.

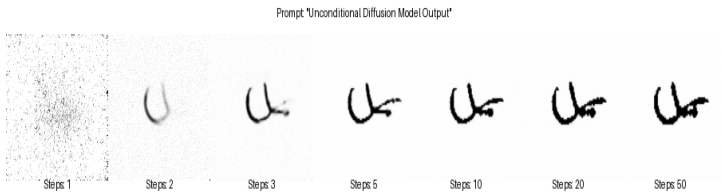
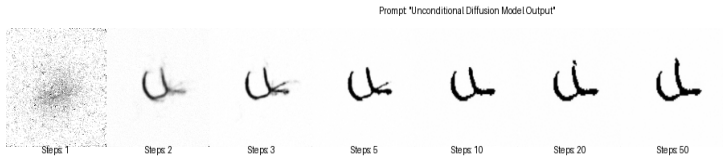
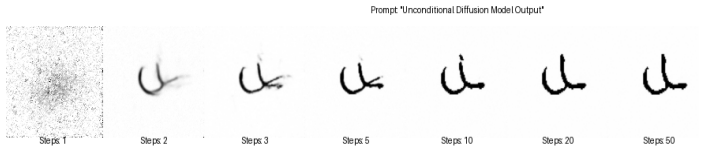
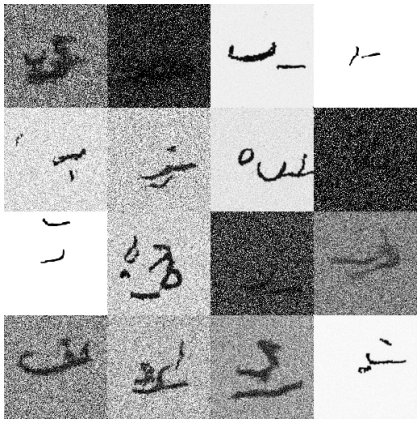
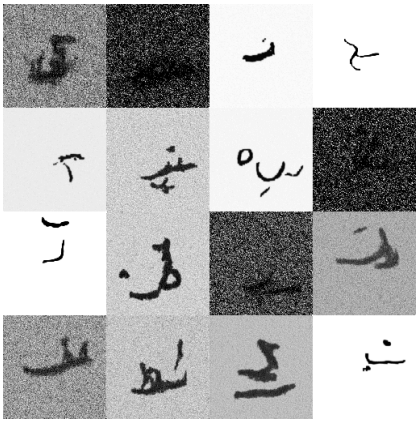
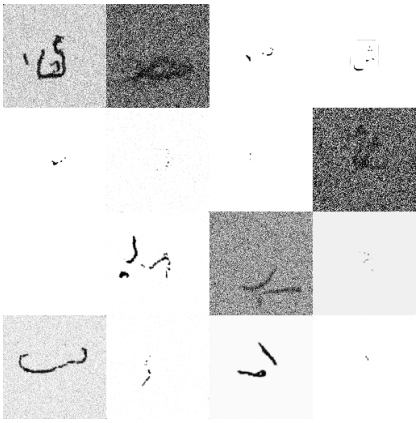
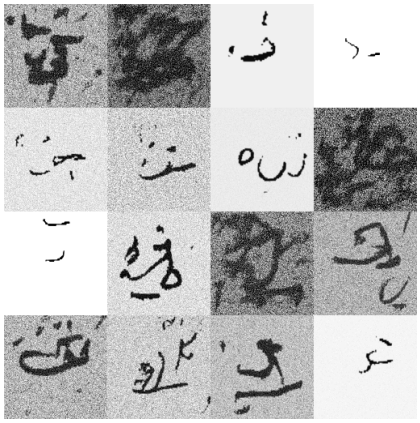
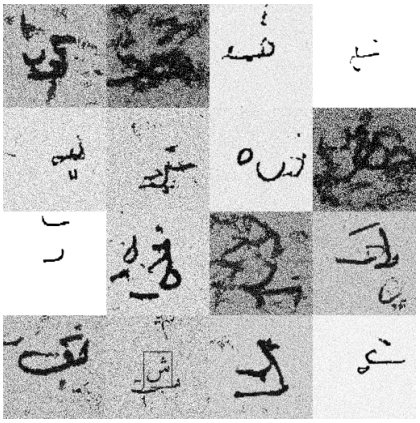
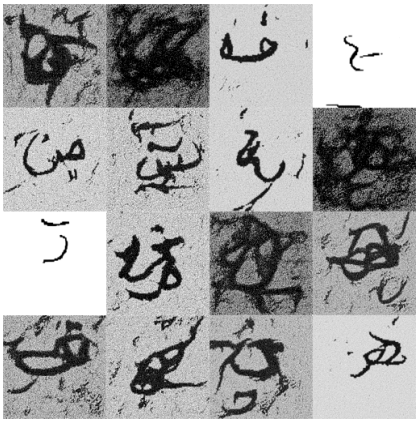
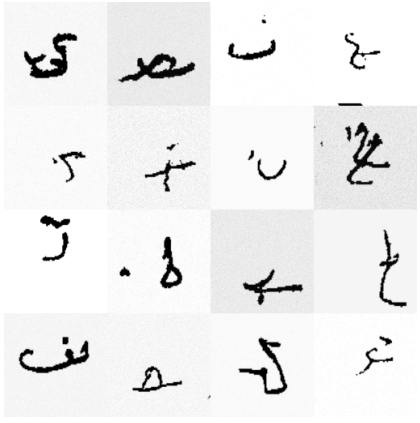
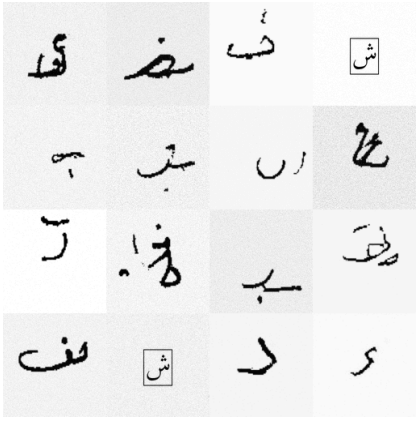
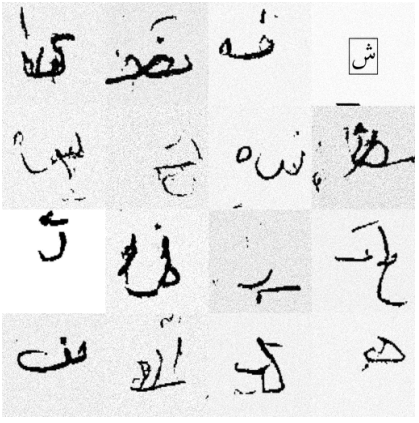
Diffusion Configuration	**Denoising Process across 50 epochs**	**Reference Image -** Arabic Character **Sad Final**
Dual-Attn HMBD-only		
Attention Layer on the Down and Up sampling Block Combined Dataset		
Attention Layer only on downsampling Combined Dataset		

Table 6: Denoising Process for a single character during inference

Epoch	- Dual-Attn - HMBD-only	- Dual-Attn - Combined Dataset	- Downl-Attn - Combined Dataset
1			
5			
10			

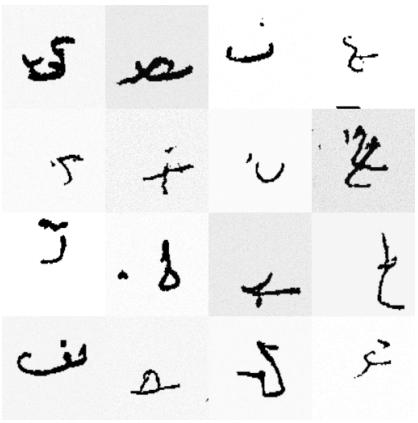
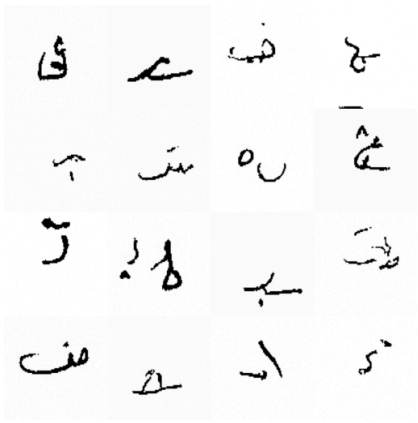
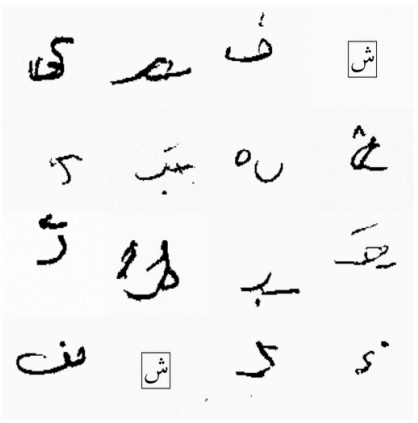
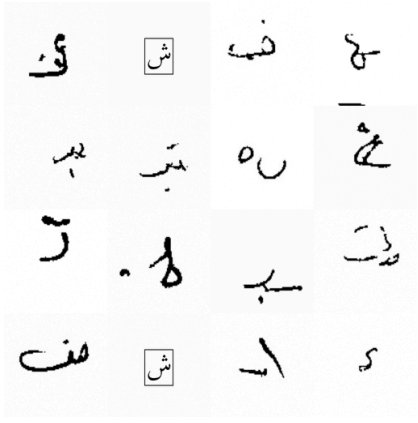
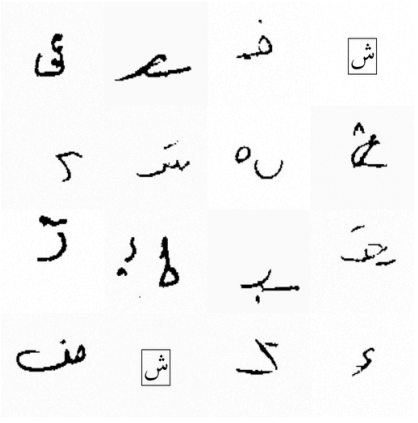
15			
19			

Table 7: Visual comparison of generated samples across training epochs. Rows indicate epochs (1, 5, 10, 15, 19); columns indicate model configurations

5.4 Convergence and Sample Quality

Throughout training, we monitored how the diffusion model's outputs evolved. In early epochs (after 1–2 epochs), generated images were mostly indecipherable blobs or random strokes. By around epoch 5, we started seeing shapes that resembled parts of Arabic letters, though often incomplete or merged with noise. The model learned more refined structures as training progressed. After about 15 epochs, most samples were clearly identifiable as Arabic characters. The training loss continued to decrease slowly even beyond this point, but the visual improvement in samples became incremental. This suggests that the model had essentially learned the major modes of the data distribution by then, and further training mainly fine-tuned the quality and cleaned up minor details.

One interesting observation was that some characters are inherently harder for the model to generate. For instance, letters with many distinct forms or those easily confused (such as *beh* (ب) vs *teh* (ت) vs *thah* (ث), which only differ by the number of dots) occasionally were mixed up by the model. We saw a few generated samples where a letter had the wrong number of dots, or an ambiguous shape that could be one of two letters. This is understandable, as the model has no explicit concept of "which letter" it is drawing (being unconditioned); it is simply replicating the distribution of all characters. This

highlights a limitation: our model sometimes produces an Arabic-style shape that might not correspond to a valid specific character, but rather an interpolation of similar characters. This happened rarely in the combined-data + attention model, but was observed a bit more in the HMBD-only model (likely due to varied handwriting causing overlaps in feature space).

We did not compute quantitative metrics like FID (Fréchet Inception Distance) for the outputs due to the scope of the project, but visually, the best model (combined data with attention) generated characters that could easily pass as new samples from the dataset. Table 7, as mentioned, provides examples of the final model's output, demonstrating both the diversity and quality achieved. To further illustrate the generation process, *Table 6* shows a sequence of intermediate images from the diffusion model denoising process for a given sample. Starting from pure noise, one can observe how faint strokes begin to emerge, then take shape as an Arabic letter (in this example, the letter is ص (sad)), and finally the image converges to a clearly readable character. This step-by-step visualization underscores the model's learned ability to construct meaningful structure out of randomness.

5.5 Summary of Findings vs. Research Questions

The experiments conducted provide clear answers to the initial research questions:

(Q1) Dataset Size and Composition: Our findings emphasize that for generating high-quality Arabic characters, **composition is more critical than sheer size**. While our combined dataset contained ~46k images, its success stemmed from merging diverse handwritten examples (~37k HMBD) with clean synthetic prototypes (~9k). This blend proved necessary to achieve both diversity and structural integrity, overcoming the limitations of HMBD-only (inconsistency) and Synthetic-only (homogeneity, poor style variation). A solely handwritten or solely synthetic dataset of similar size was insufficient.

(Q2) Performance and Quality: Diffusion models perform well on the complex Arabic script, particularly the Attention Layer on the Down and Up sampling Block for the Combined Dataset configuration. The generated symbols are visually meaningful and generally high quality, capturing the essence of different letters and exhibiting plausible variations. The primary quality limitation arises from the unconditional setup, which occasionally leads to ambiguity between letters with similar shapes but different dot patterns.

(Q3) Modifications for Better Results: We identified two key modifications that improved results. Firstly, the **dataset strategy** of combining real and synthetic data was the most impactful modification Secondly, **architectural modification** through self-attention placement was important; using attention in both downsampling and upsampling paths significantly outperformed using attention only in the downsampling path when using the combining dataset . Regarding the noise schedule, we ran out of compute credits to compare noise scheduling schements

6 Future Work

While this study demonstrates the potential of diffusion models for Arabic character generation, several promising avenues exist for future research and improvement:

- **Text-Conditioned Generation:** The most significant limitation of the current model is its unconditional nature. Implementing conditioning mechanisms is a crucial next step. By incorporating character labels (e.g., via embeddings or one-hot vectors) as conditioning input, potentially using techniques like classifier-free guidance, the model could be directed to generate specific characters and their positional forms (initial, medial, final, isolated) on demand. This would greatly enhance practical utility for applications like dataset augmentation and likely mitigate the observed ambiguity between similar glyphs.

- **Extended Training and Scaling:** Our experiments were conducted under limited computational resources (20 epochs). Extending the training duration could allow the model to further refine details and potentially eliminate minor artifacts observed in some generated samples. Additionally, exploring larger image resolutions (e.g., 128x128 or higher) and potentially increased model capacity (more U-Net layers/channels) could enable the generation of more intricate calligraphic details or the inclusion of diacritics, although this requires significantly more computational power. Utilizing more advanced sampling algorithms (e.g., DDIM, PNDM) could also optimize inference speed without sacrificing quality.

- **Architectural Optimization:** While attention proved beneficial, further architectural exploration could yield improvements or efficiencies. Investigating the precise impact of attention head dimensions or alternative attention mechanisms (e.g., linear attention) might be valuable. Furthermore, given the relative simplicity of isolated characters compared to natural scenes, exploring whether a more lightweight U-Net (e.g., fewer blocks, reduced channel counts) could achieve comparable results is warranted. This aligns with recent research trends like FreeU aimed at optimizing diffusion U-Nets.

- **Style and Context Conditioning:** Extending the model beyond isolated characters offers rich possibilities. Conditioning on style attributes (e.g., specific fonts, handwritten vs. print, calligraphic styles) could enable targeted font synthesis or handwriting simulation. Furthermore, training the model to generate characters within the context of connected script (ligatures, multi-character sequences) would be a significant step towards generating realistic Arabic text fragments.

Addressing these areas would build upon the foundation established in this work, advancing the capabilities of generative models for the nuanced complexities of Arabic and other intricate writing systems.

7 Conclusion

This project successfully demonstrated the application of diffusion probabilistic models for generating synthetic images of Arabic characters. Addressing the challenges posed by the script's inherent complexity and variability, we investigated key factors influencing generation quality, namely dataset composition and architectural design, providing insights relevant to the initial research questions guiding this work.

Our primary findings indicate that **(answering Q1)** a carefully composed dataset merging real handwritten examples (HMBDv1) with diverse synthetic font renderings is crucial for achieving high-quality results, offering a better balance of authenticity and structural consistency than either data source alone. The study confirms **(answering Q2)** that diffusion models, specifically a U-Net based DDPM, can perform well on this task, generating visually meaningful and diverse character images that capture the essence of Arabic script. However, the unconditional nature of the model presents limitations, occasionally producing ambiguous outputs between similar characters. Critically, **(answering Q3)**, we found that architectural modifications matter: incorporating self-attention mechanisms within the U-Net significantly improves output fidelity, with attention applied in both downsampling and upsampling paths yielding the most substantial benefit compared to downsampling-only attention. The standard linear noise schedule proved sufficient for this task.

While the achieved results are promising, the limitations, particularly the lack of explicit generation control, highlight clear directions for future work. Implementing text-conditioning, pursuing longer training regimes, and exploring further architectural or style-based variations hold significant potential. Nevertheless, the current work establishes a solid baseline, validating diffusion models as a powerful tool for generating complex language symbols. The insights gained regarding data strategies and architectural choices for Arabic script can inform future research in generative typography, synthetic data creation for OCR, and AI-driven tools for diverse linguistic applications. We hope this research serves as a valuable contribution and foundation for continued exploration at the intersection of generative AI and computational linguistics.

8 References

1. **Balaha, H.M., Ali, H.A., Saraya, M., et al.** (2021). *A new Arabic handwritten character recognition deep learning system (AHCR-DLS). *Neural Computing & Applications, 33*, 6325–6367. (Introduces the HMBD v1 dataset and discusses challenges in Arabic OCR.)*

2. **Ho, J.**, Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS 2020 . *(Proposed the DDPM approach for high-quality image synthesis.)*

3. **Dhariwal, P.**, & Nichol, A. (2021). *Diffusion Models Beat GANs on Image Synthesis*. arXiv:2105.05233 . *(Showed improved diffusion model architecture (with attention) outperforms GANs on image generation benchmarks.)*

4. **Encyclopaedia Britannica**. *Arabic alphabet*. (n.d.). . *(Describes the Arabic writing system and the positional forms of letters.)*

5. **Dandekar, R. A.** (n.d.). Lecture 17 Video Recording: Chinese Character Diffusion Model Generation. *Generative AI Fundamentals, Vizuara.ai*. Retrieved from https://vizuara.ai/courses/generative-ai-fundamentals/lesson/lecture-17-video-recording-chinese-character-diffusion-model-generation/

6. **Vizuara.ai.** (n.d.). *Diffusion Models from Generative AI Fundamentals.*

7. **Wu, Y. (2025)**. *A visual guide to how diffusion models work. Yue Here. Retrieved from https://yue-here.com/posts/diffusion/*